

Neural Network from Scratch

Implementation and Optimisation Pipeline

What I implemented

- Neural network built entirely from scratch using NumPy
- Forward pass, ReLU activation, and linear output layer
- Backpropagation for gradient computation
- Gradient descent parameter updates
- Mean Squared Error (MSE) loss for regression

Data and model setup

- Superconductivity dataset (**21,263 samples**)
- **81 numerical input features**
- Target variable: **critical_temperature (Tc)**
- Train/Test split
- Feature standardisation before training

Main computation

`loss = np.mean((y - y_hat)**2)`

MSE objective minimised during training

Model Pipeline:

Input Data (X, y)



Train / Test Split



Feature Scaling (Standardisation)



NN Forward Pass $\hat{y} = f(X; \theta)$



$L = \text{mean}((y - \hat{y})^2)$



Backpropagation: Compute gradients ($\nabla_{\theta} L$)



Gradient Descent Update: $\theta \leftarrow \theta - \alpha \nabla_{\theta} L$

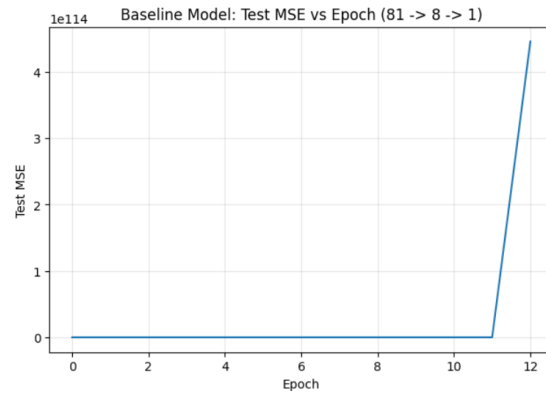


Evaluation (Test MSE & R^2)

Experimental Results and Training Behaviour

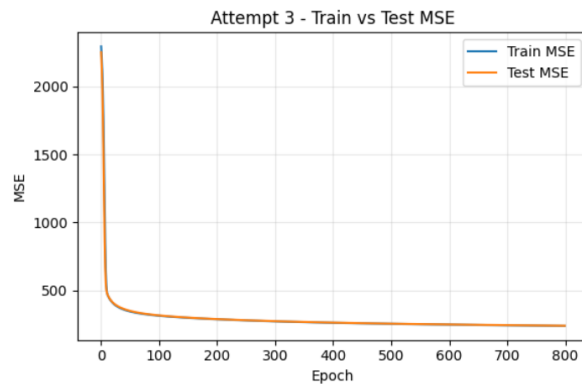
Attempt 1 -- Baseline Neural Network

- Architecture: $81 \rightarrow 8 \rightarrow 1$
- Learning Rate: 0.01
- Observed: Training diverged (loss diverged — NaN values)



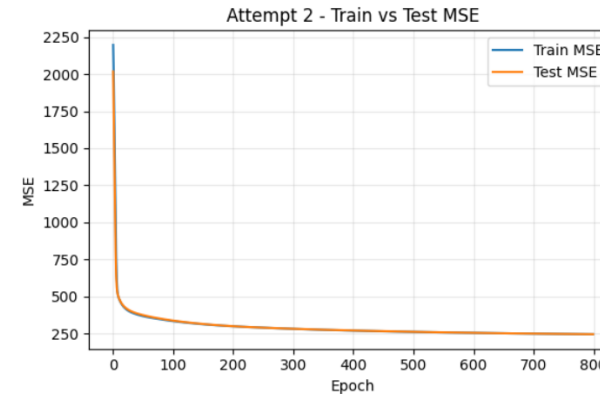
Attempt 3 -- Larger Hidden Layer

- Architecture: $81 \rightarrow 16 \rightarrow 1$
- Learning Rate: 0.001
- Observed: Best overall performance with lower test loss



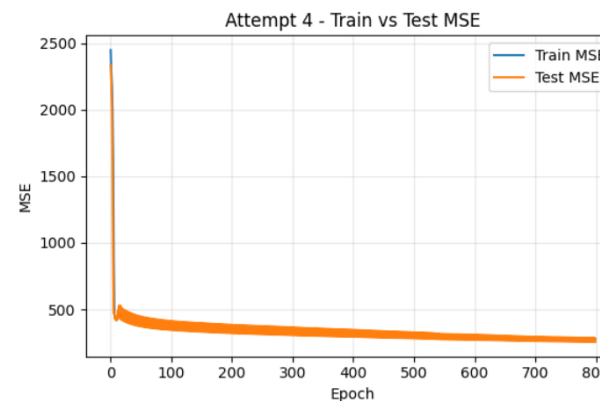
Attempt 2 -- Smaller Learning Rate

- Architecture: $81 \rightarrow 8 \rightarrow 1$
- Learning Rate: 0.001
- Observed: Stable convergence and smooth training behaviour



Attempt 4 -- Deeper Network (Increased Depth)

- Architecture: $81 \rightarrow 16 \rightarrow 8 \rightarrow 1$
- Learning Rate: 0.001
- Observed: Increased complexity did not improve generalisation



! Reducing the learning rate stabilised training, while increasing depth did not improve performance.

Neural Network Optimisation and Final Model Selection

Performance Behaviour and Design Trade-offs

Attempt	Architecture	LR	Test MSE	Test R ²
1	81→8→1	0.01	NaN	NaN
2	81→8→1	0.001	244.73	0.795
3	81→16→1	0.001	242.47	0.796
4	81→16→8→1	0.001	256.38	0.785

Training Objective:
 $\text{Loss} = \text{MSE} = \text{mean}((y - \hat{y})^2)$
Parameter Update:
 $\theta \leftarrow \theta - \alpha \nabla_{\theta} L$

Final Model Summary

Dataset:

21,263 samples | 81 features | Target: Tc

Selected model:

Attempt 3 (81 → 16 → 1)
Learning rate = 0.001

Performance:

Test MSE = 242.47
Test R² = 0.796 → model explains 79.6% of
Tc variance
RMSE = $\sqrt{242.47} \approx 15.6$ (Tc units)

Key Observations

- Lower learning rate (Attempt 1 → 2) stabilised optimisation and prevented divergence.
- Increasing hidden width (Attempt 2 → 3) produced a small but consistent improvement in Test MSE.
- Adding depth (Attempt 4) increased model complexity without improving generalisation.

Model Complexity vs Stability

- Larger models increase capacity but also optimisation difficulty.
- Simpler architectures provided more stable and reliable training.